...

Match 11, 2026

Foundations of Python Programming

Assignment 07

Working with Classes

Introduction

The goal was to extend the course registration program by introducing a Person class with validated properties for first and last name, and a Student class that inherits from Person and adds a course_name property. Both classes use Python's @property decorator for getters and setters with simple validation, and both override the __str__() method to return comma-separated output. The FileProcessor and IO classes from Assignment 06 were updated to work with Student objects instead of plain dictionaries.

Creating the Python Program

For this assignment, I am continuing to use PyCharm, the Integrated Development Environment created by JetBrains. The script written in this document follows the same template used in previous assignments:

1. Script header
2. Setup code
3. Class and Function Definitions
4. Main body

1. Script Header

Like every Python script in this course, the file begins with a script header (Figure 1.1) that includes the title, a short description of the program, and a change log.

```python
# ------------------------------------------------------------------------------------------ #
# Title: Assignment07
# Desc: This assignment demonstrates using data classes
# with structured error handling
# Change Log: (Who, When, What)
#   x,1/1/2026,Created Script
#   x,3/11/2026,Modified script to meet requirements of assignment 7.
# ------------------------------------------------------------------------------------------ #
```

Figure 1.1: Python Script Header.

2. Setup Code

The setup code section defines two constants that the rest of the program depends on: the MENU multi-line string and the FILE_NAME string (Figure 2.1). These values do not change at any point during the program, so using constants makes the script easier to maintain. If the file name ever needs to change, it only needs to be updated in one place.

This section is the same as in Assignment 06 because the storage format and file name have not changed. FILE_NAME still points to "Enrollments.json".

```python
# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data
    3. Save data to a file
    4. Exit the program
----------------------------------------
'''
FILE_NAME: str = "Enrollments.json"

# Define the Data Variables
students: list = []  # a table of student data
menu_choice: str = ""  # Hold the choice made by the user.
```

Figure 2.1: Setup code with a multi-line string block.

3. Class and Function Definitions

The program uses four classes to organize its logic (Figure 3.1). The two new classes are Person and Student, which define the data structure for enrollment records. FileProcessor contains the functions that read from and write to the JSON file. IO contains the functions that handle all user input and screen output.

```python
# Person Class
class Person:...


# Student Class (inherits from Person)
class Student(Person):...


# Processing ------------------------------------- #
class FileProcessor:...


# Presentation ------------------------------------- #
class IO:...
```

Figure 3.1: Define classes.

The Person Class

The Person class (Figure 3.2) defines two properties: first_name and last_name. Both default to an empty string. Each property uses the @property decorator for its getter, which returns the value formatted with title case using Python's .title() method. Each setter validates the input using .isalpha(), raising a ValueError if the value contains non-alphabetic characters. The class also overrides __str__() to return the first and last name as a comma-separated string.

```python
class Person:  1 usage
    """
    A class representing person data.

    Properties:
        first_name (str): The student's first name.
        last_name (str): The student's last name.

    ChangeLog:
        - x,3.11.2026,Created Class
    """
    def __init__(self, first_name: str = '', last_name: str = ''):...

    @property  # (Use this decorator for the getter or accessor)
    def first_name(self):...

    @first_name.setter
    def first_name(self, value: str):...

    @property
    def last_name(self):...

    @last_name.setter
    def last_name(self, value: str):...

    def __str__(self) -> str:
        """ Overrides the default __str__() method to return a comma-separated string.

        ChangeLog:
            - x,3.11.2026,Created method
        """
        return f'{self.first_name},{self.last_name}'
```

Figure 3.2: The Person class.

The Student Class

The Student class (Figure 3.3) inherits from Person. Its constructor calls super().__init__() to pass first_name and last_name to the Person (superclass) constructor, then sets its own course_name property. The course_name setter checks that the value is not an empty or blank string. The __str__() method overrides the one inherited from Person to include course_name as a third comma-separated field.

```python
class Student(Person):
    A class representing student data, inheriting from the Person class.

    Properties:
        first_name (str): Inherited from Person.
        last_name (str): Inherited from Person.
        course_name (str): The name of the course the student is enrolled in.

    ChangeLog:
        - x,3.11.2026,Created Class
    """

    def __init__(self, first_name: str = '', last_name: str = '', course_name: str = ''):...

    @property
    def course_name(self):...

    @course_name.setter
    def course_name(self, value: str):...

    def __str__(self) -> str:
        """ Overrides the Person __str__() method to include course_name.

        ChangeLog:
            - x,3.11.2026,Created method
        """
        return f'{self.first_name},{self.last_name},{self.course_name}'
```

Figure 3.3: The Student class.

The FileProcessor Class

The FileProcessor class was updated to work with Student objects instead of plain dictionaries.

The IO Class

The IO class was updated to create Student objects instead of dictionaries.

4. Main Body

The program defines two global variables and reads the JSON file into students on startup before entering the main loop (Figure 4.1). Because read_data_from_file() now converts each dictionary into a Student object, the students list contains typed objects from the moment the program starts.

```python
# When the program starts, read the file data into a list of Student objects
students = FileProcessor.read_data_from_file(file_name=FILE_NAME, student_data=students)
```

Figure 4.1: Startup file read.

```
Coding Assignments/Module 07/A07 via 🐍 v3.14.3
❯ python Assignment07.py


---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data
    3. Save data to a file
    4. Exit the program
----------------------------------------


Enter your menu choice number: 2
--------------------------------------------------
Vic,Vu,Python 100
Sue,Jones,Python 100
John,Doe,Python 400
Jane,Smith,Python 500
--------------------------------------------------


---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data
    3. Save data to a file
    4. Exit the program
----------------------------------------


Enter your menu choice number: 1
Enter the student's first name: Johnny
Enter the student's last name: Smith
Please enter the name of the course: Python 200

You have registered Johnny Smith for Python 200.


---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data
    3. Save data to a file
    4. Exit the program
----------------------------------------


Enter your menu choice number: 2
--------------------------------------------------
Vic,Vu,Python 100
Sue,Jones,Python 100
John,Doe,Python 400
Jane,Smith,Python 500
Johnny,Smith,Python 200
--------------------------------------------------


---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data
    3. Save data to a file
    4. Exit the program
----------------------------------------


Enter your menu choice number: 3
--------------------------------------------------
Vic,Vu,Python 100
Sue,Jones,Python 100
John,Doe,Python 400
Jane,Smith,Python 500
Johnny,Smith,Python 200
--------------------------------------------------


---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data
    3. Save data to a file
    4. Exit the program
----------------------------------------


Enter your menu choice number: 4
Program Ended
```

Figure 4.2: Testing in the terminal after testing in Pycharm..

Summary

In this assignment, I updated the course registration program to use data classes and inheritance by adding a Person class and a Student class. The FileProcessor and IO classes were updated to work with Student objects instead of plain dictionaries.

Through this assignment, I learned how to use the @property decorator to create getters and setters in Python, and how class inheritance works by having Student extend Person using super(). I also learned how overriding __str__() makes it easy to control how an object is printed without changing the code that calls it. Overall, this assignment helped me understand how data classes make programs easier to read and maintain by keeping data and validation logic together in one place.

Github repository: https://github.com/karluw26/IntroToProg-Python-Mod07/ (external link)